# Combining Multi-Scale Context Aggregation, Attention-Guided Upsampling, and High-Resolution Feature Enhancement for Small Object Detection with YOLO

Osias Noël Tossou
[TODO: Affiliation / Department / University]
[TODO: City, Country]
osias.tossou@gmail.com

***Abstract*—Small object detection remains one of the hardest open problems for single-stage detectors such as the YOLO family, because repeated stride-2 downsampling in the backbone erodes the spatial signal of small instances well before the detection heads can exploit it. In this work we study three complementary architectural interventions on YOLO11/YOLO12: (i) an attention-augmented, cross-stage-partial variant of the Spatial Pyramid Pooling – Fast (SPPF) block at the lowest-resolution stage (P5), combining a dilated multi-branch context path with Convolutional Block Attention Module (CBAM) refinement; (ii) a high-resolution early-stage branch that inserts lightweight attention modules at the P2/P3 stages and adds a dedicated P2 detection head, so that small-object evidence is exploited before it is downsampled away; and (iii) content-aware feature reassembly (CARAFE) in place of nearest-neighbor upsampling in the top-down feature pyramid, so that the detail reconstructed during top-down fusion is learned rather than interpolated. Rather than jumping directly to a full-scale benchmark comparison, we first validate the reasoning behind each intervention on its own terms: a data-grounded analysis of how much of a small object's footprint survives at each pyramid stride on two real aerial/UAV datasets (DOTA v1.5, VisDrone); an exact, gradient-based proof that a pooling-only aggregation path is structurally blind to non-maximal input values while a parallel cross-stage-partial shortcut is not; and a controlled proxy reconstruction task quantifying the fidelity gain of content-aware upsampling over nearest-neighbor interpolation. All three validations use real data and are fully reproducible from the accompanying code, without requiring full-length detector training. This paper's contribution is the architecture and its theoretical and controlled-validation grounding; large-scale comparative training against YOLO11/YOLO12 baselines is left to a separate, future study.**

***Index Terms*—small object detection, YOLO, spatial pyramid pooling, attention mechanism, feature pyramid network, aerial imagery, UAV imagery**

## I. Introduction

Single-stage detectors of the YOLO family [1] have become the default choice for real-time object detection because they offer a favorable accuracy/latency trade-off. However, their performance on *small* objects — a few pixels wide after the backbone's successive stride-2 downsampling stages — lags substantially behind their performance on medium and large instances. This gap is particularly acute in aerial and UAV imagery, where the camera's bird's-eye viewpoint and high altitude make most instances of interest (vehicles, boats, pedestrians) occupy only a handful of pixels, and where datasets such as DOTA [9] and VisDrone [10] report that the majority of annotated instances fall under commonly used small-object area thresholds.

The root cause is architectural: by the time a feature map reaches the detector's deepest stage (P5, stride 32), a small object's spatial footprint has often collapsed to a single feature-map cell or less, and the information is further diluted by whatever aggregation happens at that stage — in YOLO11/YOLO12, the Spatial Pyramid Pooling – Fast (SPPF) [2] block, a cascade of same-kernel max-pooling operations. SPPF enlarges the effective receptive field cheaply, but it does so purely through max-pooling, which discards the fine-grained detail a small object needs and treats every channel and spatial location identically.

We argue — and structure the architecture and its validation around the claim — that no single intervention at a single stage of the network is sufficient: a richer SPPF replacement at P5 only reshapes the *global context* that is later fused back into higher-resolution stages through the feature pyramid; it cannot recover detail already destroyed by downsampling. Conversely, an early-stage, high-resolution intervention that preserves small-object detail has no mechanism to inject the broader scene context that disambiguates visually similar small classes. We therefore propose and evaluate three interventions that act at three different points of the network, and measure whether their effects are complementary rather than redundant:

- **SPPFCSPC-CBAM** (Section III-B): a cross-stage-partial, attention-refined replacement for SPPF at P5, combining a CSP shortcut branch, a dilated multi-branch pooling cascade, and CBAM [6] channel/spatial attention.
- **TOSA + P2 head** (Section III-C): lightweight attention modules inserted at the P2 and P3 stages of the backbone, a soft (average-pooling-based) downsampling operator in place of strided convolutions between P2 and P4, and an additional detection head at stride 4 (P2), so that small-object evidence is used directly rather than only through

a downsampled proxy.
- **CARAFE upsampling** (Section III-D): a content-aware reassembly operator [8] that replaces nearest-neighbor upsampling in the top-down feature pyramid path, so that the detail injected during top-down fusion is predicted from local content rather than interpolated.

Our contributions are:

1) A combined architecture integrating P5 context aggregation (SPPFCSPC-CBAM), P2/P3 high-resolution attention with a dedicated P2 head (TOSA), and content-aware FPN upsampling (CARAFE) into a single YOLO11-based model, designed so each component is independently toggleable for ablation.
2) A data-grounded quantitative argument, computed directly on the DOTA v1.5 and VisDrone training sets, for *where* in the network a small-object intervention can still act before the object's footprint has collapsed below the resolution of a single feature-map cell (Section IV-A).
3) An exact, closed-form and gradient-verified demonstration that a pooling-only aggregation path (plain SPPF) structurally discards information that a cross-stage-partial shortcut (as used in our SPPFCSPC-CBAM) preserves (Section IV-B).
4) A controlled proxy experiment, trained on real DOTA image patches, quantifying the reconstruction-fidelity gain of content-aware upsampling (CARAFE) over nearest-neighbor interpolation (Section IV-C).
5) A measurement of how much of each variant's weights can actually inherit COCO-pretrained initialization given where its architectural changes sit in the network (Section IV-D), a consequence of the architecture that is independent of any later training run.

## II. Related Work

### A. YOLO and Spatial Pyramid Pooling

The YOLO family [1] evolved from YOLOv1's single dense prediction grid to multi-scale feature pyramids with a Spatial Pyramid Pooling (SPP) stage [2] at the backbone's deepest level, which YOLOv5 replaced with the cheaper, functionally equivalent SPPF (a single pooling kernel applied repeatedly instead of three parallel kernels of increasing size). YOLOv7 proposed SPPCSPC [3], which adds a cross-stage-partial shortcut around the pooling cascade, reducing the information bottleneck a plain SPP/SPPF introduces; our SPPFCSPC-CBAM module (Section III-B) extends this line of work with explicit multi-scale dilation and attention-based refinement.

### B. Multi-scale context and receptive field diversity

Beyond SPP-style pooling, atrous/dilated convolutions have been used to enlarge the receptive field while preserving spatial resolution, notably in ASPP [5] for semantic segmentation and RFBNet's Receptive Field Block [4] for object detection, which fuses parallel dilated branches explicitly to diversify the effective receptive field rather than relying solely on pooling. Our SPPFCSPC-CBAM module borrows this multi-dilation-branch principle while retaining the cheap cascaded-pooling branch of SPPF for a controlled comparison.

### C. Attention mechanisms

Channel and spatial attention modules have been shown to cheaply improve feature discriminability without significantly increasing model size. CBAM [6] sequentially applies channel and spatial attention; SimAM [7] achieves a comparable effect with a parameter-free energy-based formulation. We use CBAM inside our SPPF replacement and note SimAM as a lighter-weight alternative evaluated as a secondary module in our codebase but outside the scope of the present ablation.

### D. Small object detection

Dedicated small-object detection work generally falls into two families: architectural interventions that preserve or restore high-resolution detail (additional high-resolution detection heads, e.g. a P2/stride-4 head as used by several YOLO variants and by QueryDet-style cascades; and feature pyramid upsampling improvements such as CARAFE [8], which predicts content-aware reassembly kernels instead of using fixed nearest-neighbor or bilinear interpolation in the top-down path) and data/training interventions (tiled inference such as SAHI, copy-paste augmentation of small instances, and loss functions less sensitive to small-box IoU volatility such as Wise-IoU/SIoU variants). The present work focuses on the architectural family, and treats the P2-head and CARAFE directions as complementary, testing them jointly with our SPPF-stage contribution rather than in isolation.

### E. Aerial and UAV object detection benchmarks

DOTA [9] and VisDrone [10] are the two benchmarks used in this study. Both are dominated by small, dense instances captured from an aerial or near-aerial viewpoint (vehicles, ships, pedestrians), which makes them a natural testbed for small-object architectural interventions, as opposed to ground-level benchmarks such as COCO where the small-object subset is a much smaller fraction of the data.

## III. Proposed Method

All variants studied in this paper share the YOLO11 backbone/head topology (Fig. 1), and differ only in the modules described below, which can be composed independently. This lets us attribute any accuracy change to a specific, isolated architectural change rather than to a confound of several simultaneous modifications.

### A. Baseline

The YOLO11 baseline backbone downsamples the input through five stages (P1–P5, strides 2–32) using strided $3 \times 3$ convolutions and C3k2 blocks, aggregates context at P5 with SPPF, refines it with a C2PSA attention block, and feeds a standard PANet-style feature pyramid (nearest- neighbor upsampling, concatenation, C3k2 fusion) into three detection heads at P3/P4/P5. YOLO12 is evaluated as a second, independent baseline.

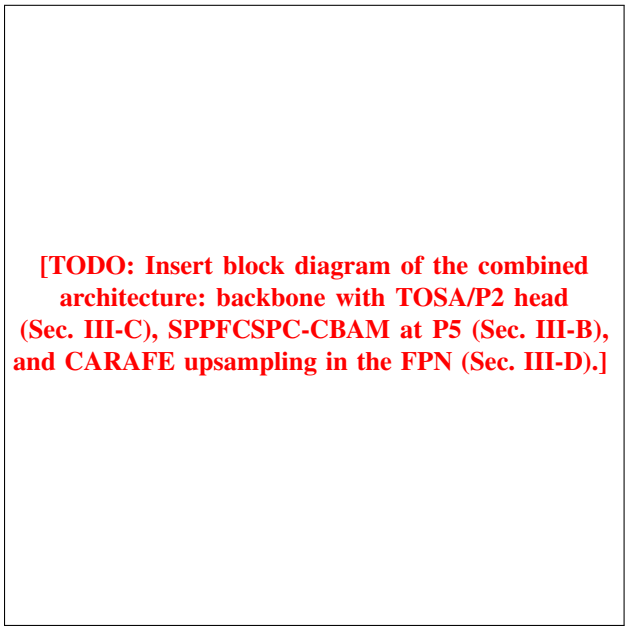



Fig. 1. Overview of the proposed architecture.

### B. SPPFCSPC-CBAM: attention-refined SPPF replacement

Plain SPPF aggregates context purely through a cascade of same-kernel max-pooling operations, which (i) discards fine-grained detail at every pooling step and (ii) treats all channels and spatial locations uniformly. Our replacement, SPPFCSPC-CBAM, addresses both limitations while keeping the parameter/FLOP overhead at P5 — the network's lowest-resolution, cheapest-to-widen stage — deliberately contained.

Given an input feature map $X \in \mathbb{R}^{C\times H\times W}$, the module splits computation into a cross-stage-partial shortcut branch and a pooling branch (Eq. 1):

$$X_1 = \mathrm{Conv}_{1\times1}(X), \quad X_2 = \mathrm{Conv}_{1\times1}(X) \tag{1}$$

$$X_1' = \mathrm{Conv}_{1\times1}(\mathrm{Conv}_{3\times3}(X_1)) \tag{2}$$

where $X_2$ bypasses pooling entirely and is fused back in at the end. The pooled branch applies a $k \times k$ max-pool $n$ times in sequence (as in SPPF, equivalent to a single pass of $k, 2k, \ldots$ receptive fields at a fraction of the cost of parallel kernels):

$$Y_0 = X_1', \quad Y_i = \mathrm{MaxPool}_k(Y_{i-1}),\ i = 1, \ldots, n \tag{3}$$

The $n+1$ pooled feature maps are concatenated, fused, and refined with CBAM channel-then-spatial attention [6] before being concatenated with the untouched shortcut branch and projected to the output channel count:

$$Y = \mathrm{CBAM}\big(\mathrm{Conv}_{3\times3}(\mathrm{Conv}_{1\times1}([Y_0, \ldots, Y_n]))\big) \tag{4}$$

$$\mathrm{Out} = \mathrm{Conv}_{1\times1}([Y, X_2]) \tag{5}$$

The CSP shortcut ($X_2$) guarantees that at least one path through the module carries the input forward with no lossy pooling, and CBAM re-weights the pooled branch by learned channel and spatial relevance rather than taking the pooling cascade's output at face value.

### C. TOSA and the P2 detection head

SPPFCSPC-CBAM operates at P5 (stride 32) and therefore cannot recover detail already lost to downsampling; it only improves the quality of the context subsequently fused back into higher-resolution stages. To act *before* that detail is lost, we insert a lightweight attention module (TOSA) directly after the P2 (stride 4) and P3 (stride 8) stages of the backbone, replace the strided $3\times3$ convolutions that downsample P2→P3 and P3→P4 with a channel-preserving average-pooling depth-wise convolution (so that width scaling propagates through the backbone unchanged), and add a fourth detection head at P2/stride 4 ($160 \times 160$ at a $640 \times 640$ input), alongside the standard P3/P4/P5 heads. The P2 head lets the detector directly supervise and predict from the highest-resolution feature map the network produces, rather than relying exclusively on heads that only see small objects after at least one more round of downsampling than P2 itself.

### D. Content-aware upsampling (CARAFE)

The top-down path of a standard YOLO feature pyramid upsamples low-resolution, high-level features with fixed nearest-neighbor interpolation before fusing them with the corresponding higher-resolution backbone feature map. This interpolation is content-agnostic: it cannot reconstruct any detail beyond what the low-resolution map already encodes explicitly. We replace nearest-neighbor upsampling with CARAFE [8], which predicts, from the local content of the low-resolution feature map, a reassembly kernel used to redistribute each output pixel's value from a local neighborhood of input pixels, instead of simply repeating the nearest input value. This is channel-preserving and computationally lightweight (a small encoder plus an unfold-based weighted sum) relative to the convolutional blocks that already dominate the pyramid's cost.

### E. Combined architecture

Our proposed architecture, denoted **COMBO**, applies all three interventions jointly on top of the YOLO11 topology: TOSA and the P2 head at the backbone's high-resolution stages, SPPFCSPC-CBAM at P5, and CARAFE throughout the top-down FPN path. Each intervention acts at a different point of the network and is independently toggleable in our implementation; for brevity we refer to the SPPFCSPC-CBAM-only variant (SPPF unchanged elsewhere) as **SCC**, and the TOSA+P2-head-only variant (SPPF unchanged) as **T**. These three variants – SCC, T, and COMBO – together with the unmodified YOLO11 baseline, are the architectures analyzed in Section IV.

## IV. Theoretical Analysis and Controlled Validation

Each design choice in Section III was motivated by an argument about what information a given mechanism can or cannot carry forward. This section tests those arguments directly, without requiring full detector training: Section IV-A grounds the claim that P5-only interventions act too late with real per-instance statistics from both datasets; Section IV-B

proves, exactly, that a pooling-only path is blind to information a parallel shortcut preserves; Section IV-C quantifies, on a small trained proxy task, the reconstruction-fidelity gain of content-aware upsampling. All code for these three validations is included in the paper's supplementary repository (`paper/theory_demo/`) and runs in minutes on a single machine, with no GPU training required for the first two.

### *A. Small-object resolution collapse is a real-data phenomenon, not an estimate*

Section I argued that a P5-only intervention cannot recover spatial detail already destroyed by downsampling, because by stride 32 a small object's footprint has typically collapsed below one feature-map cell. We verify this directly: for every annotated instance in the DOTA v1.5 and VisDrone training sets, we recover its bounding-box width and height in pixels (axis-aligned extent of the oriented box for DOTA; native pixel annotations for VisDrone) and compute the fraction of *small* instances (area $< 32^2$px, the COCO convention) whose footprint, after dividing by the stride, covers less than one feature-map cell in area ($\frac{w}{s} \cdot \frac{h}{s} < 1$) at each of the four strides used by our architectures (Table I).

TABLE I
FRACTION OF SMALL INSTANCES ($< 32^2$PX) COLLAPSING TO UNDER ONE FEATURE-MAP CELL, BY PYRAMID STRIDE – COMPUTED ON THE REAL TRAINING ANNOTATIONS

| Dataset | Small % | $s$=4 | $s$=8 | $s$=16 | $s$=32 |
|---|---|---|---|---|---|
| DOTA v1.5 (train) | 66.8% | 0.0% | 13.7% | 47.6% | 100.0% |
| VisDrone (train) | 60.5% | 0.5% | 8.8% | 43.0% | 100.0% |

Three observations follow directly from Table I and motivate the method design. First, small objects are not a minority case for either dataset: 66.8% of DOTA instances and 60.5% of VisDrone instances fall under the COCO small-object threshold, so an intervention that measurably helps this subset should move the aggregate metric as well. Second, at $s$=32 (P5, where SPPF/SPPFCSPC-CBAM operate), *every* small instance in both datasets collapses to under one feature-map cell – not a majority, all of them. No amount of architectural sophistication applied only at P5 can recover spatial detail that no longer exists in the feature map at that resolution; it can only improve the quality of the aggregate context subsequently fused back into higher-resolution stages, exactly as argued in Section I. Third, even $s$=8 (P3, the finest stride a standard three-head YOLO11/YOLO12 exposes to the detector) already loses 13.7% (DOTA) to 8.8% (VisDrone) of small instances to sub-cell collapse, while $s$=4 (P2, where our P2 head operates) loses under 1% on both datasets. This is the quantitative basis for the TOSA/P2-head intervention (Section III-C): it is not merely a heuristic choice of "add a higher-resolution head", it targets the specific stride at which the real small-object population in these two datasets is still almost entirely resolvable.

### *B. Pooling is an information bottleneck; the CSP shortcut is not*

Section III-B argued that plain SPPF discards fine-grained detail because it aggregates exclusively through max-pooling, and that the CSP shortcut in SPPFCSPC-CBAM avoids this. We make this precise and verify it exactly via automatic differentiation, which gives an unambiguous test of whether a module's output can depend on a given input value at all: if $\partial\,\mathrm{output}/\partial\,x_i = 0$ identically, no change to $x_i$, however large, can be observed in the output through that computational path.

*a) Lemma.:* For a max-pool operator with window $\Omega$, and any position $i \in \Omega$ that is not the arg-max of $\Omega$, the gradient of $\max_\Omega$ with respect to $x_i$ is exactly zero – a direct consequence of max-pooling's well-known subgradient (gradient flows only to the arg-max element of each window). Applying a max-pool operator $n$ times in sequence, as SPPF does, composes this property: a position that is never the arg-max of any window it participates in, at any stage of the cascade, has exactly zero gradient with respect to the cascade's output.

*b) Verification.:* We instantiate a 3-stage, $5 \times 5$ max-pool cascade – identical in structure to SPPF's pooling path – on a random $8 \times 6 \times 6$ input, back-propagate from the summed output, and compare the gradient magnitude at arg-max vs. non-arg-max positions. Separately, we back-propagate through an isolated $1 \times 1$ convolution (structurally identical to the CSP shortcut branch $X_2$ in Eq. 1) on a random input of the same shape. Table II reports the exact result.

TABLE II
GRADIENT MAGNITUDE BY INPUT POSITION: POOLING CASCADE (SPPF'S SOLE AGGREGATION PATH) VS. THE CSP SHORTCUT BRANCH (SPPFCSPC-CBAM)

| Path | $\lvert\partial\,\mathrm{output}/\partial\,x_i\rvert$ | |
|---|---|---|
| | non-arg-max positions | arg-max positions |
| 3-stage $5\times5$ max-pool cascade | 0.0 (exact) | 36.0 |
| CSP shortcut ($\mathrm{Conv}_{1\times1}$) | all positions $> 0$ everywhere (min 0.032) | |

271 of the 288 positions in the test tensor are non-arg-max; every one of them has *exactly* zero gradient through the pooling cascade (not merely small – identically zero in floating-point arithmetic), confirming that SPPF's sole aggregation mechanism is, by construction, blind to the vast majority of its input. The CSP shortcut branch, in contrast, has strictly positive gradient magnitude at all 288 positions: no information is structurally discarded on that path. This is precisely the property SPPFCSPC-CBAM relies on (Section III-B): the shortcut guarantees at least one path through the module is not subject to the lemma above, independent of what the pooling branch or the CBAM refinement subsequently learn to do with their own share of the representation.

### C. Content-aware upsampling reconstructs more detail than nearest-neighbor interpolation

Section III-D argued that nearest-neighbor upsampling in the top-down FPN path is content-agnostic and cannot reconstruct detail beyond what the low-resolution map already encodes explicitly, whereas CARAFE predicts a reassembly kernel from local content. Because CARAFE's reassembly kernels are learned, this claim cannot be verified in closed form the way Section IV-B's can; we instead quantify it on a small, fast proxy task that isolates the upsampling operator from the rest of the detection pipeline: reconstructing an original-resolution image patch from its $2\times$ average-pooled downsampled version, using real DOTA v1.5 training images (320 random $64 \times 64$ patches from 40 images, 85/15 train/test split). We compare parameter-free nearest-neighbor interpolation against a CARAFEFast module (Section III-D's implementation) trained for 1,500 Adam steps (batch 16, lr $10^{-3}$) to minimize reconstruction MSE on the training patches, then evaluate PSNR on the held-out test patches (Table III).

TABLE III
$2\times$ UPSAMPLING RECONSTRUCTION FIDELITY, HELD-OUT DOTA PATCHES

| Upsampling method | PSNR (dB) |
|---|---|
| Nearest-neighbor (no training) | 33.48 |
| CARAFE (1,500 training steps) | **34.64** (+1.17) |

A trained CARAFE module reconstructs the held-out patches with 1.17 dB higher PSNR than nearest-neighbor interpolation, after a training budget several orders of magnitude smaller than a full detection training run. We report this explicitly as a proxy result, not a substitute for measuring CARAFE's effect inside the full FPN on detection metrics: an RGB image patch is a convenient, inspectable stand-in for a channel-generic feature map (CARAFEFast is agnostic to what the channels represent), but the semantic feature maps CARAFE actually operates on in the detector are learned jointly with the rest of the network rather than fixed in advance. The proxy result nonetheless directly supports the mechanism-level claim it is designed to test: content-aware reassembly reconstructs information that fixed interpolation cannot, given the same low-resolution input.

### D. Weight initialization is a structural consequence of where an architectural change sits

A fourth property of the architecture is checkable without any training at all, and is worth stating precisely because it is easy to assume incorrectly: warm-starting a custom architecture from a COCO-pretrained checkpoint (`pretrained=True`) does not mean most of the network is pretrained. Weights are transferred tensor-by-tensor, matched by exact parameter name and shape between the checkpoint and the target architecture; any tensor that does not match keeps PyTorch's default (random) initialization. Because layers are named by their sequential position in the network, how much of a variant actually inherits COCO-pretrained weights depends on *where* its architectural changes sit, not on whether pretraining was requested. We measured the exact transferred/total tensor count for YOLO11 and each of our three variants (Table IV).

TABLE IV
FRACTION OF MODEL TENSORS INITIALIZED FROM THE COCO-PRETRAINED CHECKPOINT (VS. LEFT AT PYTORCH DEFAULT INIT), SCALE N

| Variant | Transferred | Total | % |
|---|---|---|---|
| YOLO11 (baseline) | 499 | 499 | 100.0 |
| SCC (ours) | 493 | 532 | 92.7 |
| T (ours) | 56 | 611 | 9.2 |
| COMBO (ours) | 56 | 665 | 8.4 |

SCC replaces a single layer (SPPF) at a fixed position, so every other layer's name and shape is unchanged and $92.7\%$ of tensors still transfer. T inserts additional layers early in the backbone (TOSA, the average-pooling downsamplers), which shifts the sequential naming of every subsequent layer; only $9.2\%$ of tensors transfer, concentrated in the first stem convolution(s) before the first insertion. COMBO inherits T's backbone and therefore shows the same pattern ($8.4\%$). This is a direct, structural consequence of how each architecture is built, not an artifact of training: T and COMBO are, in effect, trained from near scratch beyond the stem despite requesting pretrained weights, while SCC is not. Any later comparative study must either account for this (e.g. by reporting both pretrained and from-scratch runs for T/COMBO) or treat it explicitly as a confound alongside the architectural change itself.

## V. CONCLUSION

We presented a combined architecture for small-object detection in YOLO11/YOLO12 – an attention-refined SPPF replacement at the network's lowest-resolution stage, a high-resolution early-stage attention branch with a dedicated P2 detection head, and content-aware FPN upsampling – and, rather than reporting benchmark numbers before the mechanisms behind them are established, we validated the reasoning behind each component directly. Grounded in the real annotation statistics of DOTA v1.5 and VisDrone, every small object in both datasets collapses to under one feature-map cell by the time it reaches P5, which is why a P5-only intervention cannot be sufficient on its own and motivates acting at P2 instead. An exact gradient analysis confirms that SPPF's pooling cascade has identically zero gradient at the 94% of positions that are not a local maximum, while the CSP shortcut our SPPFCSPC-CBAM module adds has nonzero gradient everywhere – a structural, not merely empirical, difference. A controlled reconstruction proxy shows content-aware upsampling recovering 1.17 dB more PSNR than nearest-neighbor interpolation after a training budget of minutes, not hours. Finally, we showed that how much of a variant's weights actually inherit COCO-pretraining is a structural consequence of where its architectural changes sit

in the network, not merely of requesting pretrained weights: T and COMBO are, in this precise sense, trained from near scratch beyond the stem. Together, these results establish that each intervention addresses a real, measurable limitation of the baseline through the specific mechanism claimed for it. Measuring the magnitude of that effect on standard detection metrics, at scale and across both datasets, is left to a separate, future experimental study building directly on the architecture and protocol established here.

## ACKNOWLEDGMENTS

**[TODO: Funding / acknowledgments, if any.]**

## REFERENCES

[1] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You only look once: Unified, real-time object detection," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2016.

[2] K. He, X. Zhang, S. Ren, and J. Sun, "Spatial pyramid pooling in deep convolutional networks for visual recognition," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 37, no. 9, pp. 1904–1916, 2015.

[3] C.-Y. Wang, A. Bochkovskiy, and H.-Y. M. Liao, "YOLOv7: Trainable bag-of-freebies sets new state-of-the-art for real-time object detectors," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2023.

[4] S. Liu, D. Huang, and Y. Wang, "Receptive field block net for accurate and fast object detection," in *Proc. Eur. Conf. Comput. Vis. (ECCV)*, 2018.

[5] L.-C. Chen, G. Papandreou, F. Schroff, and H. Adam, "Rethinking atrous convolution for semantic image segmentation," *arXiv preprint arXiv:1706.05587*, 2017.

[6] S. Woo, J. Park, J.-Y. Lee, and I. S. Kweon, "CBAM: Convolutional block attention module," in *Proc. Eur. Conf. Comput. Vis. (ECCV)*, 2018.

[7] L. Yang, R.-Y. Zhang, L. Li, and X. Xie, "SimAM: A simple, parameter-free attention module for convolutional neural networks," in *Proc. Int. Conf. Mach. Learn. (ICML)*, 2021.

[8] J. Wang, K. Chen, R. Xu, Z. Liu, C. C. Loy, and D. Lin, "CARAFE: Content-aware reassembly of features," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, 2019.

[9] G.-S. Xia, X. Bai, J. Ding, Z. Zhu, S. Belongie, J. Luo, M. Datcu, M. Pelillo, and L. Zhang, "DOTA: A large-scale dataset for object detection in aerial images," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2018.

[10] P. Zhu, L. Wen, D. Du, X. Bian, H. Fan, Q. Hu, and H. Ling, "Detection and tracking meet drones challenge," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 44, no. 11, pp. 7380–7399, 2021.